Libraries

In [5]:
```
import pandas as pd
import numpy as np
from pathlib import Path
import seaborn as sns
import matplotlib.pyplot as plt
```

Load & Configure

In [6]:
```
# Configure
sns.set_theme(style="whitegrid")
```

In [7]:
```
def find_project_root(marker="requirements.txt"):
    current = Path.cwd().resolve()
    for parent in [current, *current.parents]:
        if (parent / marker).exists():
            return parent
    raise RuntimeError(f"Could not find project root (no {marker} found)")

PROJECT_ROOT = find_project_root()
proc  = pd.read_parquet(PROJECT_ROOT / "data" / "processed" / "patio_features.parquet")
print(f"Loaded processed features: {proc.shape}")
```

```
Loaded processed features: (87696, 42)
```

Feature Validation

In [8]:
```
# Validate the merge
print("ONI columns present:", [c for c in proc.columns if c in ('oni', 'enso_phase')])
print(f"Missing ONI values: {proc ['oni'].isnull().sum()}")
print(f"ONI range in our window: {proc ['oni'].min():.1f} to {proc['oni'].max():.1f}")
print(f"\nENSO phase distribution (hours) in our 5-year window:")
print(proc ['enso_phase'].value_counts())
```

```
ONI columns present: ['oni', 'enso_phase']
Missing ONI values: 0
ONI range in our window: -1.0 to 2.1

ENSO phase distribution (hours) in our 5-year window:
enso_phase
neutral    39360
la_nina    29280
el_nino    19056
Name: count, dtype: int64
```

Lag features validation

In [9]:
```python
# Lag features validation

expected_lag_features = [
    'precip_lag1', 'precip_lag3', 'precip_sum_3h',
    'gust_lag1', 'gust_max_3h',
    'pressure_msl_trend_3h', 'pressure_msl_trend_6h',
    'cloud_cover_low_lag1', 'humidity_lag1', 'humidity_lag3',
]
for f in expected_lag_features:
    assert f in proc.columns, f"Missing lag feature: {f}"
print(f"All {len(expected_lag_features)} lag features present.\n")

# Spot-check: precip_lag1 at row T should equal precipitation at row T-1 (same location)
kingston = proc[proc['location'] == 'kingston'].sort_values('time').reset_index(drop=True)
sample = kingston.head(8)[['time', 'precipitation', 'precip_lag1', 'precip_lag3', 'precip_sum_3h']]
print("First few rows of Kingston (showing NaN boundary behavior):")
print(sample.to_string(index=False))
```

```
All 10 lag features present.

First few rows of Kingston (showing NaN boundary behavior):
                 time  precipitation  precip_lag1  precip_lag3  precip_sum_3h
2021-05-01 00:00:00            0.0          NaN          NaN            NaN
2021-05-01 01:00:00            0.0          0.0          NaN            NaN
2021-05-01 02:00:00            0.0          0.0          NaN            NaN
2021-05-01 03:00:00            0.0          0.0          0.0            0.0
2021-05-01 04:00:00            0.0          0.0          0.0            0.0
2021-05-01 05:00:00            0.0          0.0          0.0            0.0
2021-05-01 06:00:00            0.0          0.0          0.0            0.0
2021-05-01 07:00:00            0.0          0.0          0.0            0.0
```

In [10]:
```python
# Verify boundary NaN counts per location
print("NaN counts per location:")
for col, expected in [('precip_lag1', 1), ('precip_lag3', 3),
                      ('pressure_msl_trend_6h', 6), ('precip_sum_3h', 3)]:
    nan_counts = proc.groupby('location')[col].apply(lambda x: x.isna().sum())
    print(f"  {col} (expect {expected} per location):")
    for loc, n in nan_counts.items():
        print(f"    {loc}: {n}")
```

```
NaN counts per location:
  precip_lag1 (expect 1 per location):
    kingston: 1
    montego_bay: 1
  precip_lag3 (expect 3 per location):
    kingston: 3
    montego_bay: 3
  pressure_msl_trend_6h (expect 6 per location):
    kingston: 6
    montego_bay: 6
  precip_sum_3h (expect 3 per location):
    kingston: 3
    montego_bay: 3
```

In [11]:
```python
# Verify no future leakage by reconstructing one lag manually
ks = proc[proc['location'] == 'kingston'].sort_values('time').reset_index(drop=True)
row_t = ks.iloc[100]
row_t_minus_1 = ks.iloc[99]
row_t_minus_3 = ks.iloc[97]
```

```
assert row_t['precip_lag1'] == row_t_minus_1['precipitation'], "precip_lag1 mismatch"
assert row_t['precip_lag3'] == row_t_minus_3['precipitation'], "precip_lag3 mismatch"

# precip_sum_3h at T = precip[T-1] + precip[T-2] + precip[T-3], NOT including T
expected = ks.iloc[97:100]['precipitation'].sum()  # rows 97, 98, 99
assert abs(row_t['precip_sum_3h'] - expected) < 1e-9, "precip_sum_3h mismatch"

# Pressure trend = current - past, so order matters
expected_trend = row_t['pressure_msl'] - row_t_minus_3['pressure_msl']
assert abs(row_t['pressure_msl_trend_3h'] - expected_trend) < 1e-9, "pressure trend mismatch"

print("All lag-feature spot checks passed — no future leakage, correct boundaries.")
```

```
All lag-feature spot checks passed — no future leakage, correct boundaries.
```

Target validation

Confirms `build_target()` produced a target consistent with the EDA's definition.

In [12]:
```
# Existence and type
assert "wet_veranda" in proc.columns, "wet_veranda missing from processed data"
assert proc["wet_veranda"].dtype.kind == "i", "wet_veranda should be int"

# Overall rate matches EDA (Section 12.3): 9.18%
overall_rate = proc["wet_veranda"].mean() * 100
print(f"Overall positive rate: {overall_rate:.2f}% (EDA: 9.18%)")

# By-location split matches EDA (Section 12.5)
by_loc = proc.groupby("location")["wet_veranda"].mean().mul(100).round(2)
print(f"\nBy location:\n{by_loc}")
print("Expected: kingston ~6.00%, montego_bay ~12.37%")
```

```
Overall positive rate: 9.18% (EDA: 9.18%)

By location:
location
kingston        6.00
montego_bay    12.37
Name: wet_veranda, dtype: float64
Expected: kingston ~6.00%, montego_bay ~12.37%
```